

Gramática regular — PromptZal

Proyecto 1 — Lenguajes Formales y de Programación

1. Definición formal

$$G = (N, T, P, S)$$

- **N** (no terminales): un no terminal por cada estado del AFD que participa en el reconocimiento de tokens: `{S0, S1, S2, S2b, S2c, S3, S3b, S4, S5, S6, S6a, S6b, S6c}`
- **T** (terminales): los símbolos concretos de entrada. Por brevedad se agrupan en clases (igual que en el AFD), y cada clase representa una familia de producciones, una por símbolo concreto que contiene:
 - `letra` representa cualquiera de `a...z, A...Z`
 - `digito` representa cualquiera de `0...9`
 - símbolos individuales: `@ " { } ( ) = + - > . / * _` y el carácter `\n`
 - `cualquiera` representa cualquier símbolo del alfabeto salvo los que se excluyan explícitamente en esa producción (p. ej. "cualquiera $\neq$ " y $\neq$ \n" dentro de `S4`)
- **S** (símbolo inicial): `S0`
- **P**: el conjunto de producciones, listado en la sección 3, por rama. Todas tienen la forma `A → aB` o `A → a` — no se usa `A → λ` en ningún punto, porque ningún lexema válido de PromptZal es la cadena vacía.

2. Regla de derivación usada

Para cada transición $\delta(A, a) = B$ del AFD:

- Se agrega **siempre** $A \rightarrow aB$
- Si B es un estado de aceptación (produce token o cierra un comentario/directiva válidamente), se agrega **también** $A \rightarrow a$, permitiendo terminar la derivación justo en ese punto.

Esto es lo que genera automáticamente todas las longitudes válidas de un lexema (por ejemplo, identificadores de 1, 2, 3... caracteres) sin necesitar λ .

3. Producciones por rama

Identificadores y palabras reservadas (vía S1)

```
S0 → letra S1
S0 → letra           (identificador de una sola letra)
S1 → letra S1 | digito S1 | _ S1
S1 → letra      | digito      | _
```

La distinción identificador vs. palabra reservada no es sintáctica — ambas se derivan igual. Se resuelve después, comparando el lexema completo contra la tabla de palabras reservadas, tal como en el AFD.

Números — enteros y decimales (vía S2, S2b, S2c)

S0 → digito S2
S0 → digito (entero de un solo dígito)
S2 → digito S2
S2 → digito
S2 → . S2b (no hay S2 → . sola: S2b no es de aceptación)
S2b → digito S2c
S2b → digito
S2c → digito S2c
S2c → digito

Directivas (vía S3, S3b)

S0 → @ S3 (no hay S0 → @ sola: S3 no es de aceptación)
S3 → letra S3b
S3 → letra
S3b → letra S3b
S3b → letra

Cadenas de texto (vía S4)

S0 → " S4 (no hay S0 → " sola: S4 no es de aceptación por sí mismo)
S4 → cualquiera S4 (cualquiera ≠ " y ≠ \n)
S4 → " (cierre: termina el lexema CADENA)

Aquí no existe ninguna producción `S4 → \n ...`: por diseño, `\n` no tiene transición válida dentro de `S4`. Esa ausencia formal es exactamente lo que en el AFD se traduce como caída a `qE` — la gramática simplemente no genera ninguna cadena que contenga `\n` en su interior, porque el lenguaje de cadenas válidas de PromptZal no las admite.

Flecha -> (vía S5)

S0 → - S5	(no hay S0 → - sola: "-" no es válido por sí mismo)
S5 → >	(cierre: termina el lexema FLECHA)

Comentario de línea (vía S6, S6a)

S0 → / S6	(no hay S0 → / sola: S6 no es de aceptación por sí mismo)
S6 → / S6a	
S6 → /	(comentario vacío "/" es válido: S6a es de aceptación)
S6a → cualquiera S6a	(cualquiera ≠ \n)
S6a → cualquiera	

Comentario de bloque (vía S6, S6b, S6c)

S6 → * S6b	(no hay S6 → * sola: un bloque mínimo requiere "/*/")
S6b → cualquiera S6b	(incluye \n; no es de aceptación)
S6b → * S6c	
S6c → * S6c	
S6c → /	(cierre: termina el comentario de bloque)
S6c → cualquiera S6b	(cualquier otro carácter regresa al cuerpo del comentario)

Símbolos de un carácter (directo desde S0, sin no terminal intermedio)

S0 → =	(ASIGNACION)
S0 → +	(CONCATENACION)
S0 → {	(LLAVE_ABRE)
S0 → }	(LLAVE_CIERRA)
S0 → (	(PAR_ABRE)
S0 →)	(PAR_CIERRA)

4. Nota sobre alcance

Alcance de la gramática: esta gramática describe el lenguaje de lexemas válidos (lo que sí se reconoce y genera token, o que se descarta correctamente como comentario). No incluye producciones para las rutas de error del AFD (las que caen en `qE`): un error léxico es, por definición, una cadena que **no** pertenece a `L(G)` — eso es justo lo que hace que el Lexer lo detecte y lo reporte, en vez de derivarlo desde `S0`.